

Trabajo práctico 3.1

Técnicas de diseño de algoritmos

Dividir y conquistar

1. Implemente en Python una función que dada una lista retorne el menor elemento de la misma.

```
>>mas_chico([1,3,4,5,6,7,8])
1
```
2. Implemente en Python una función que calcule la suma de los elementos de una lista.

```
>>sumar([1,2,3,4,5])
15
```
3. Sea $f(x): \mathbb{N} \rightarrow \mathbb{Z}$ una función monótona decreciente que puede ser evaluada en tiempo constante. Es decir, calcular $T_f \in O(1)$
 - a. Implemente una función en Python para determinar el valor más pequeño $x \in [a, b]$ tal que $f(x) < 0$. En este caso no utilice dividir y conquistar. Utilice un algoritmo iterativo ordinario. Si calculamos $T(n)$ para ese algoritmo, ¿qué significaría n ?
 - b. Calcule el $O(T(n))$ para la función anterior.
 - c. Implemente una función en Python que calcule lo mismo que la anterior pero ahora utilizando la estrategia de dividir y conquistar.
 - d. Calcule la complejidad temporal de la función anterior (O-grande).
4. **Secuencia.** Dada la siguiente secuencia:

$$s_k = \begin{cases} 1 & \text{si } 1 \leq k \leq 3 \\ 2s_{k-1} + 3s_{k-2} + 4s_{k-3} & \text{si } k > 3 \end{cases}$$

- a. Implemente una función recursiva en Python que calcule s_k
- b. Calcule el $O(T(n))$. Para hacerlo, tenga en cuenta que
$$3T(n-1) > T(n-1) + T(n-2) + T(n-3)$$

Programación dinámica

5. Implemente una función que calcule el término k de la sucesión de Fibonacci aplicando la estrategia de programación dinámica.
6. Utilice la estrategia de programación dinámica para resolver el ejercicio 4.
7. **Problema de la mochila binaria.** Dado:
 - Un conjunto $S = \{e_1, e_2, e_3, \dots, e_k\}$ de k elementos
 - Una función $v: S \rightarrow \mathbb{N}$ que asigna a cada e_i un peso.
 - Un peso máximo W .

Implemente un algoritmo que encuentre un conjunto máximo M de elementos tales que la suma de los pesos no supere W .